DESARROLLO DE APIS EN .NET

APIs en .NET

Conceptos fundamentales · Apunte completo de estudio

HTTP · HTTPS · REST · SOAP · Verbos · JSON · Endpoints · Headers · JWT
Lambda · Clases · Interfaces · Servicios · Controladores · Program.cs
Entity Framework · DbContext · Migraciones · LINQ · Inyección · Middleware · CORS

Instituto Técnico Superior Córdoba

Programación III · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	HTTP y la web · Protocolo, HTTPS, verbos, estados, headers
02	REST, SOAP, endpoints y JSON
03	Autenticación con JWT
04	C# esencial para APIs
05	Arquitectura de una API en .NET
06	Persistencia con Entity Framework Core
07	Validación, errores, middleware y CORS

01 · HTTP y la web

HTTP — HyperText Transfer Protocol

HTTP es el protocolo de comunicación que define cómo se intercambian mensajes entre un **cliente** (navegador, app, Postman) y un **servidor** web. Cada vez que hacés una petición a una API, usás HTTP. Es un protocolo **sin estado**: cada request es independiente, el servidor no recuerda los anteriores.

Un mensaje HTTP tiene dos partes: el *request* (lo que el cliente envía) y el *response* (lo que el servidor devuelve).

Estructura de un REQUEST y un RESPONSE HTTP

```
// REQUEST HTTP
GET /api/productos/5 HTTP/1.1
Host: miapi.com
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json
// El body va aquí (en GET generalmente está vacío)

// RESPONSE HTTP
HTTP/1.1 200 OK
Content-Type: application/json
{
    "id": 5,
    "nombre": "Teclado",
    "precio": 5000
}
```

HTTPS — HTTP Secure

HTTPS es exactamente lo mismo que HTTP, pero con una capa de **cifrado TLS** (antes llamado SSL) por encima. Los datos viajan encriptados entre el cliente y el servidor: nadie en el medio puede leer ni modificar lo que se transmite.

Característica	HTTP	HTTPS
Puerto por defecto	80	443
Datos cifrados	No — viajan en texto plano	Sí — cifrado TLS
Certificado necesario	No	Sí (SSL/TLS cert)
Uso recomendado	Solo desarrollo local	Siempre en producción
URL	http://...	https://...

> **Tip:** En .NET, la línea `app.UseHttpsRedirection()` en `Program.cs` redirige automáticamente cualquier request HTTP al equivalente HTTPS.

Verbos HTTP

Los verbos (o métodos HTTP) indican la **intención** de la petición. En una API REST cada verbo tiene un significado semántico específico relacionado con operaciones CRUD (Crear, Leer, Actualizar, Eliminar).

Verbo	Uso y comportamiento
GET	Obtener datos. No modifica nada en el servidor. Puede repetirse sin efectos secundarios (idempotente).
POST	Crear un nuevo recurso. Envía datos en el body. Cada llamada crea un recurso nuevo.
PUT	Reemplazar un recurso completo. Envía el objeto entero. Si no existe, puede crearlo.
PATCH	Actualizar parcialmente un recurso. Solo envía los campos que cambian.
DELETE	Eliminar un recurso. Generalmente no tiene body.

Verbos en un controlador .NET

```
[HttpGet]            public async Task<IActionResult> GetTodos() { }
[HttpGet("{id}")]    public async Task<IActionResult> GetById(int id) { }
[HttpPost]           public async Task<IActionResult> Crear([FromBody] Producto p) { }
[HttpPut("{id}")]    public async Task<IActionResult> Reemplazar(int id, Producto p) { }
[HttpPatch("{id}")]  public async Task<IActionResult> Actualizar(int id, ...) { }
[HttpDelete("{id}")] public async Task<IActionResult> Eliminar(int id) { }
```

Códigos de estado HTTP

Son números de 3 dígitos que el servidor incluye en cada response para indicar el resultado de la operación. El primer dígito agrupa la familia: **2xx** = éxito, **3xx** = redirección, **4xx** = error del cliente, **5xx** = error del servidor.

Código / Método .NET	Cuándo usarlo
200 Ok()	Todo salió bien. Retorna datos.
201 Created()	Se creó un nuevo recurso exitosamente.
204 NoContent()	Éxito pero sin datos que retornar (ej: DELETE).
400 BadRequest()	El cliente envió datos inválidos o mal formados.
401 Unauthorized()	No está autenticado (falta token o es inválido).

Código / Método .NET	Cuándo usarlo
403 Forbid()	Está autenticado pero no tiene permiso.
404 NotFound()	El recurso solicitado no existe.
409 Conflict()	Conflicto con el estado actual (ej: email duplicado).
422 UnprocessableEntity()	Validación de negocio fallida.
500 StatusCode(500)	Error interno del servidor (bug no controlado).

Headers HTTP

Los headers son **metadatos** que viajan junto con cada request y response HTTP. Están separados del body y proveen información sobre el contenido, el cliente, la autenticación y el formato esperado, entre otros.

Header	Qué indica
Content-Type: application/json	El body que envío está en formato JSON.
Accept: application/json	El cliente espera recibir una respuesta en JSON.
Authorization: Bearer <token>	Token JWT para autenticar al usuario.
Cache-Control: no-cache	No cachear esta respuesta.
X-Request-ID: abc-123	ID único del request (para trazabilidad/logs).

Leer y agregar headers en .NET

```
// Leer un header del request entrante
var token = Request.Headers["Authorization"].ToString();

// Agregar un header a la response
Response.Headers.Append("X-Version", "1.0");

// En el cliente (HttpClient)
httpClient.DefaultRequestHeaders.Add("Authorization", $"Bearer {token}");
```

02 · REST, SOAP, endpoints y JSON

REST — Representational State Transfer

REST es un **estilo de arquitectura** para diseñar APIs. No es un protocolo ni un estándar: es un conjunto de principios y convenciones sobre cómo organizar los endpoints, usar los verbos HTTP y estructurar las respuestas. Una API que sigue estos principios se llama **RESTful**.

Principio	Qué significa en la práctica
Sin estado (Stateless)	Cada request contiene toda la info necesaria. El servidor no guarda sesión entre requests.
Recursos como URLs	Cada entidad es un recurso identificado por una URL única (`/productos` , `/usuarios/5`).
Verbos HTTP para acciones	GET para leer, POST para crear, PUT/PATCH para modificar, DELETE para eliminar.
Respuestas en JSON/XML	Los datos se transfieren en un formato estándar, generalmente JSON.
Interfaz uniforme	Las URLs siguen convenciones consistentes en toda la API.

Diseño de URLs RESTful para una API de productos

```
GET    /api/productos      // obtener todos
GET    /api/productos/5    // obtener el producto con id=5
POST   /api/productos      // crear un producto nuevo
PUT    /api/productos/5    // reemplazar el producto id=5
PATCH  /api/productos/5    // modificar parcialmente id=5
DELETE /api/productos/5    // eliminar el producto id=5

// Recursos anidados (relaciones)
GET    /api/categorias/3/productos  // productos de la categoría 3
```

SOAP — Simple Object Access Protocol

SOAP es un **protocolo formal** para intercambiar mensajes entre sistemas, usando XML como formato de datos. A diferencia de REST (que es un estilo), SOAP define reglas estrictas: cómo se estructura el mensaje, cómo se manejan los errores y cómo se describe el servicio (mediante WSDL). Es más complejo pero ofrece contratos formales y soporte nativo para transacciones y seguridad avanzada.

Ejemplo de mensaje SOAP (XML)

```
POST /ServicioProductos HTTP/1.1
Content-Type: text/xml

<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <ObtenerProducto xmlns="http://miapi.com/">
           <ProductoId>5</ProductoId>
        </ObtenerProducto>
    </soap:Body>
</soap:Envelope>
```

Aspecto	REST	SOAP
Formato de datos	JSON (principalmente)	XML (obligatorio)
Protocolo	HTTP	HTTP, SMTP, TCP y otros
Contrato formal	Opcional (OpenAPI/Swagger)	Obligatorio (WSDL)
Rendimiento	Más liviano y rápido	Más pesado por el XML
Estándar de seguridad	HTTPS + JWT	WS-Security (más robusto)
Cuándo usarlo	APIs web modernas, móvil, microservicios	Sistemas bancarios, ERP, legados empresariales

¿Qué es un Endpoint?

Un endpoint es una **URL específica** de tu API que acepta requests. Es el punto de contacto entre el cliente y el servidor. Cada combinación de *verbo HTTP + URL* es un endpoint distinto. Se forma con: verbo HTTP + URL base + ruta del recurso + parámetros (opcionales).

Anatomía de un endpoint

```
https://api.mitienda.com/api/productos/5?incluirStock=true
// └── URL base/dominio ──┘ └ recurso ┘ id └ query parameter ┘

// En .NET, la ruta se define así:
[Route("api/[controller]")]  // -> api/productos
[HttpGet("{id}")]            // -> api/productos/{id}
```

JSON — JavaScript Object Notation

JSON es el formato de datos más usado en APIs REST. Es un texto estructurado en pares **clave-valor** que tanto humanos como máquinas pueden leer fácilmente. En .NET, la serialización y deserialización de JSON es automática gracias a `System.Text.Json`.

Tipos de datos en JSON

```
{
   "nombre": "Ana",            // string
   "edad": 28,                 // number
   "activo": true,             // boolean
   "saldo": null,              // null
   "roles": ["admin","user"],  // array
   "direccion": {              // objeto anidado
      "calle": "San Martín",
      "ciudad": "Córdoba"
   }
}

// En .NET: el controlador lo deserializa automáticamente
public async Task<IActionResult> Crear([FromBody] Producto p)
// [FromBody] lee el JSON del request y lo convierte al objeto Producto
```

03 · Autenticación con JWT

JWT (JSON Web Token) es un estándar para transmitir información de forma segura entre cliente y servidor. Es un string codificado en Base64 dividido en 3 partes separadas por puntos: **Header.Payload.Signature**. El servidor genera el token al hacer login, y el cliente lo envía en cada request siguiente.

Estructura de un JWT

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhbmEiLCJyb2wiOiJhZG1pbiJ9 . xK9sD2mP
//     HEADER        .          PAYLOAD (claims)          .  FIRMA
// algoritmo usado    usuario, roles, expiración          firmado
```

Flujo de autenticación

- **1.** El usuario hace `POST /api/auth/login` con usuario y contraseña.
- **2.** El servidor valida y devuelve un JWT.
- **3.** El cliente guarda el token y lo envía en cada request: `Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...`
- **4.** El servidor valida el token y permite o rechaza (401/403).

Proteger endpoints en .NET con JWT

Program.cs — registrar autenticación JWT

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(secretKey)),
            ValidateIssuer = false,
            ValidateAudience = false
        };
    });
```

En el controlador — requerir autenticación

```
[Authorize]                   // cualquier usuario autenticado
[Authorize(Roles = "admin")]  // solo administradores
public async Task<IActionResult> GetSecreto() { ... }

[AllowAnonymous]              // no requiere token
public async Task<IActionResult> GetPublico() { ... }
```

> **Tip:** El `401 Unauthorized` significa "no sé quién sos" (falta o es inválido el token). El `403 Forbidden` significa "sé quién sos, pero no tenés permiso para esto". Son errores distintos.

04 · C# esencial para APIs

Funciones Lambda

Una función lambda es una **función anónima**: una función sin nombre que escribís directamente donde la necesitás, sin declararla por separado. En C# se usa la sintaxis `x => expresión`, que se lee "dado x, retorná esa expresión".

Ejemplos de lambdas

```
// Lambda de un parámetro
Func<int, bool> esMayor = edad => edad >= 18;

// Lambda de dos parámetros
Func<int, int, int> sumar = (a, b) => a + b;

// Donde más las vas a ver: LINQ sobre colecciones
var adultos = usuarios
    .Where(u => u.Edad >= 18)  // filtra
    .Select(u => u.Nombre)     // proyecta
    .OrderBy(n => n)           // ordena
    .ToList();
```

Variables

Un espacio en memoria con un nombre y un tipo que guarda un valor. En C# el tipo puede ser **explícito** o **inferido** con `var`.

Declaración de variables

```
int edad = 25;               // tipo explícito
string nombre = "Ana";       // cadena de texto
bool activo = true;          // verdadero o falso
decimal precio = 99.99m;     // decimal de alta precisión

var cantidad = 10;           // el compilador infiere int
var texto = "Hola";          // el compilador infiere string

const decimal IVA = 0.21m;   // constante: nunca cambia
```

Clases

Un **molde** para crear objetos. Agrupa datos relacionados (propiedades) y el comportamiento que actúa sobre esos datos (métodos). A partir de una clase podés crear múltiples instancias (objetos).

Definición y uso de una clase

```
public class Producto {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public decimal Precio { get; set; }

    public string ObtenerDescripcion()
        => $"{Nombre} - ${Precio}";
}

// Crear una instancia
var prod = new Producto { Id = 1, Nombre = "Teclado", Precio = 5000m };
Console.WriteLine(prod.ObtenerDescripcion()); // Teclado - $5000
```

Clase estática

Una clase que **no se puede instanciar** (no se usa `new`). Existe una sola vez en toda la aplicación. Se usa para agrupar funciones utilitarias o constantes que no necesitan estado propio.

Clase estática: utilidades de precios

```
public static class PrecioHelper {
    public const decimal TASA_IVA = 0.21m;

    public static decimal AplicarIva(decimal precio)
        => precio * (1 + TASA_IVA);

    public static decimal AplicarDescuento(decimal precio, int pct)
        => precio * (1 - pct / 100m);
}

// Uso: directo, sin new
var total  = PrecioHelper.AplicarIva(5000m);           // 6050
var oferta = PrecioHelper.AplicarDescuento(6050m, 10); // 5445
```

Interfaz

Un **contrato**. Define qué métodos y propiedades debe tener una clase, pero no cómo los implementa. Permite que tu código dependa de la interfaz (el contrato) y no de una clase concreta, facilitando cambiar implementaciones sin tocar el resto del código.

Interfaz y su implementación

```
public interface IProductoRepository {
    Task<List<Producto>> ObtenerTodosAsync();
    Task<Producto?> ObtenerPorIdAsync(int id);
    Task GuardarAsync(Producto producto);
}

// Clase que firma ese contrato
public class ProductoRepository : IProductoRepository {
    private readonly AppDbContext _db;
    public ProductoRepository(AppDbContext db) => _db = db;

    public async Task<List<Producto>> ObtenerTodosAsync()
        => await _db.Productos.ToListAsync();

    public async Task<Producto?> ObtenerPorIdAsync(int id)
        => await _db.Productos.FindAsync(id);
}
```

05 · Arquitectura de una API en .NET

Una API en .NET se organiza en capas con responsabilidades bien separadas: el **controlador** recibe el HTTP, el **servicio** aplica la lógica de negocio y el **repositorio** accede a los datos.



Cada capa habla solo con la siguiente; el controlador no toca la base de datos.

Servicio

Una clase que contiene la **lógica de negocio**. El servicio no accede directamente a la base de datos (eso lo hace el repositorio) ni responde HTTP (eso lo hace el controlador). Es el intermediario que procesa y toma decisiones.

Servicio de productos

```
public class ProductoService {
    private readonly IProductoRepository _repo;
    public ProductoService(IProductoRepository repo) => _repo = repo;

    public async Task<List<Producto>> ObtenerCarosAsync() {
        var todos = await _repo.ObtenerTodosAsync();
        return todos.Where(p => p.Precio > 10000).ToList();
    }

    public async Task<Producto> CrearAsync(Producto nuevo) {
        if (nuevo.Precio <= 0)
            throw new ArgumentException("El precio debe ser positivo");
        await _repo.GuardarAsync(nuevo);
        return nuevo;
    }
}
```

Controlador

El **punto de entrada HTTP**. Recibe los requests del cliente, llama al servicio y retorna un `IActionResult` con el código HTTP apropiado. No debe tener lógica de negocio propia.

Controlador completo

```
[ApiController]
[Route("api/[controller]")]
public class ProductosController : ControllerBase {
    private readonly ProductoService _service;
    public ProductosController(ProductoService service) => _service = service;

    [HttpGet]                                   // 200
    public async Task<IActionResult> GetTodos()
        => Ok(await _service.ObtenerCarosAsync());

    [HttpGet("{id}")]                           // 404 o 200
    public async Task<IActionResult> GetById(int id) {
        var p = await _service.ObtenerPorIdAsync(id);
        return p is null ? NotFound() : Ok(p);
    }

    [HttpPost]
    public async Task<IActionResult> Crear([FromBody] Producto nuevo) {
        if (!ModelState.IsValid) return BadRequest(ModelState); // 400
        var creado = await _service.CrearAsync(nuevo);
        return CreatedAtAction(nameof(GetById),                 // 201
            new { id = creado.Id }, creado);
    }

    [HttpDelete("{id}")]                         // 204
    public async Task<IActionResult> Eliminar(int id) {
        await _service.EliminarAsync(id);
        return NoContent();
    }
}
```

Program.cs

Es el **punto de arranque** de la aplicación. Cumple dos responsabilidades: (1) registrar todos los servicios en el contenedor de inyección de dependencias, y (2) configurar el pipeline HTTP que procesa cada request entrante.

Program.cs completo

```
var builder = WebApplication.CreateBuilder(args);

// 1. REGISTRAR SERVICIOS
builder.Services.AddControllers();
builder.Services.AddScoped<IProductoRepository, ProductoRepository>();
builder.Services.AddScoped<ProductoService>();
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(/* opciones JWT */);

// 2. CONSTRUIR LA APP
var app = builder.Build();

// 3. CONFIGURAR EL PIPELINE (middleware)
app.UseHttpsRedirection();  // HTTP -> HTTPS
app.UseAuthentication();    // valida quién sos
app.UseAuthorization();     // valida qué podés hacer
app.MapControllers();       // conecta rutas a controladores
app.Run();
```

Inyección de dependencias (DI)

La inyección de dependencias es un patrón en el que una clase **no crea sus propias dependencias**: las recibe desde afuera (generalmente por el constructor). En .NET, el contenedor de DI administra la creación y el ciclo de vida de todos los servicios registrados en `Program.cs`.

MAL — acoplado, difícil de testear

```
public class ProductoService {
    private readonly ProductoRepository _repo;
    public ProductoService() {
        _repo = new ProductoRepository(); // crea la dependencia adentro
    }                                     // imposible mockear en tests
}
```

BIEN — desacoplado, testeable

```
public class ProductoService {
    private readonly IProductoRepository _repo;
    public ProductoService(IProductoRepository repo) // la recibe
        => _repo = repo;
}

// Program.cs: el contenedor sabe cómo construir todo
builder.Services.AddScoped<IProductoRepository, ProductoRepository>();
builder.Services.AddScoped<ProductoService>();
```

Ciclo de vida	Comportamiento
AddScoped	Una instancia por request HTTP. El más usado en APIs.

Ciclo de vida	Comportamiento
AddSingleton	Una instancia para toda la vida de la app.
AddTransient	Nueva instancia cada vez que se solicita.

IActionResult — ¿por qué el controlador lo devuelve?

`IActionResult` es una interfaz de ASP.NET Core que representa **cualquier tipo de respuesta HTTP**. El controlador declara que devuelve `IActionResult` y en tiempo de ejecución decide qué respuesta concreta retornar: 200 con datos, 404 si no existe, 400 si los datos son inválidos, 201 si se creó, etc.

Con IActionResult — control total

```
public async Task<IActionResult> GetById(int id) {
    var prod = await _service.ObtenerPorIdAsync(id);
    if (prod is null) return NotFound(); // 404
    return Ok(prod);                     // 200 con el objeto
}

// ActionResult<T>: variante tipada (recomendada en .NET moderno)
public async Task<ActionResult<Producto>> GetById(int id) {
    var prod = await _service.ObtenerPorIdAsync(id);
    if (prod is null) return NotFound();
    return Ok(prod); // o simplemente: return prod;
}
```

Método	Qué hace
Ok(obj)	Status 200, serializa obj a JSON en el body.
Created(url, obj)	Status 201, header Location con la URL del recurso.
NoContent()	Status 204, sin body.
BadRequest(modelState)	Status 400 con lista de errores.
NotFound()	Status 404, sin body.
Unauthorized()	Status 401.
Forbid()	Status 403.
StatusCode(500, obj)	Status personalizado con body opcional.

06 · Persistencia con Entity Framework Core

ORM — Object-Relational Mapper

Un ORM es una herramienta que traduce automáticamente entre el mundo orientado a objetos (clases C#) y el mundo relacional (tablas SQL). En lugar de escribir SQL a mano, trabajás con objetos y el ORM genera y ejecuta las consultas por vos. Esto reduce errores, acelera el desarrollo y hace el código más legible.

Sin ORM (SQL manual)	Con ORM (Entity Framework)
`string sql = "SELECT * FROM Productos WHERE Precio > 1000";` + abrir conexión, comando, lectura...	`var caros = await _db.Productos.Where(p => p.Precio > 1000).ToListAsync();`
`string sql = "INSERT INTO Productos (Nombre, Precio) VALUES (@n, @p)";` + comando...	`await _db.Productos.AddAsync(nuevo); await _db.SaveChangesAsync();`

Los ORM más usados en .NET son **Entity Framework Core** (el oficial de Microsoft) y **Dapper** (más liviano, consultas SQL explícitas).

Entity Framework Core

EF Core es el ORM oficial de .NET. Mapea clases C# a tablas de base de datos, maneja las relaciones entre entidades, genera el SQL automáticamente y administra los cambios mediante migraciones. Soporta SQL Server, PostgreSQL, MySQL, SQLite y otros motores.

Configuración básica de EF Core

```
// 1. Instalar los paquetes NuGet
// dotnet add package Microsoft.EntityFrameworkCore.SqlServer
// dotnet add package Microsoft.EntityFrameworkCore.Tools

// 2. Definir la entidad (clase = tabla)
public class Producto {
    public int Id { get; set; }               // columna Id (PK automática)
    public string Nombre { get; set; }        // columna Nombre
    public decimal Precio { get; set; }       // columna Precio
    public int CategoriaId { get; set; }      // FK a Categorias
    public Categoria Categoria { get; set; }  // navegación (JOIN)
}

// 3. Registrar en Program.cs
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
```

DbContext

El `DbContext` es la clase central de EF Core. Representa la **sesión con la base de datos** y expone los `DbSet<T>` (uno por tabla). Es a través del DbContext que hacés todas las operaciones: consultar, agregar, modificar y eliminar registros.

Definición del DbContext

```
public class AppDbContext : DbContext {
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    // Cada DbSet<T> representa una tabla
    public DbSet<Producto> Productos { get; set; }
    public DbSet<Categoria> Categorias { get; set; }
    public DbSet<Usuario> Usuarios { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder) {
        // Relación: un Producto tiene una Categoría
        modelBuilder.Entity<Producto>()
            .HasOne(p => p.Categoria)
            .WithMany(c => c.Productos)
            .HasForeignKey(p => p.CategoriaId);

        // Configurar precisión de decimal
        modelBuilder.Entity<Producto>()
            .Property(p => p.Precio)
            .HasColumnType("decimal(18,2)");
    }
}
```

Migraciones

Las migraciones son archivos de código que describen **cómo evoluciona el esquema** de la base de datos a medida que cambia tu modelo. EF Core las genera automáticamente comparando tus clases con el estado actual de la BD. Funcionan como un control de versiones para la base de datos.

Flujo de trabajo con migraciones

```
// 1. Modificás una clase (ej: agregás una propiedad)
public class Producto {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public decimal Precio { get; set; }
    public string Descripcion { get; set; } // <- propiedad nueva
}

// 2. Generás la migración (desde la terminal)
dotnet ef migrations add AgregarDescripcionAProducto

// 3. Aplicás la migración a la BD
dotnet ef database update
// EF ejecuta: ALTER TABLE Productos ADD Descripcion nvarchar(max)

// Otros comandos útiles
dotnet ef migrations list              // ver historial
dotnet ef database update NombreAnterior  // revertir
```

LINQ con Entity Framework

LINQ (Language Integrated Query) es la sintaxis de consultas integrada en C#. Cuando la usás sobre un `DbSet` de EF Core, las lambdas se traducen automáticamente a SQL. Esto significa que el filtrado ocurre **en la base de datos**, no en memoria — mucho más eficiente.

LINQ → SQL: EF Core traduce las lambdas

```
// WHERE  ->  SQL: WHERE Precio > 1000
var caros = await _db.Productos
    .Where(p => p.Precio > 1000)
    .ToListAsync();

// SELECT (proyección)  ->  SQL: SELECT Id, Nombre
var nombres = await _db.Productos
    .Select(p => new { p.Id, p.Nombre })
    .ToListAsync();

// JOIN (navegación con Include)  ->  SQL: LEFT JOIN Categorias
var conCategoria = await _db.Productos
    .Include(p => p.Categoria)
    .Where(p => p.Categoria.Nombre == "Tech")
    .ToListAsync();

// ORDER BY + paginación  ->  OFFSET 20 ROWS FETCH NEXT 10
var pagina = await _db.Productos
    .OrderBy(p => p.Nombre)
    .Skip(20).Take(10)
    .ToListAsync();

// FirstOrDefault / Count / Any
var prod   = await _db.Productos.FirstOrDefaultAsync(p => p.Id == id);
var total  = await _db.Productos.CountAsync();
var existe = await _db.Productos.AnyAsync(p => p.Nombre == "Teclado");
```

Métodos más usados de EF Core

> **Importante — SaveChangesAsync():** EF Core trabaja con un sistema de seguimiento de cambios (change tracker). Los métodos `Add`, `Update` y `Remove` **no ejecutan nada** en la base de datos de forma inmediata — solo marcan la entidad en memoria. Recién cuando llamás `SaveChangesAsync()` EF genera y ejecuta el SQL real.

Métodos de consulta y de modificación

```
// --- CONSULTA ---
var prod = await _db.Productos.FindAsync(id);          // por PK, el más eficiente
var p1   = await _db.Productos.FirstOrDefaultAsync(p => p.Nombre == "Teclado");
var p2   = await _db.Productos.SingleOrDefaultAsync(p => p.Codigo == "ABC"); // excepción si hay >1
var todos = await _db.Productos.ToListAsync();
decimal suma = await _db.Productos.SumAsync(p => p.Precio);

// --- MODIFICACIÓN (nada se ejecuta hasta SaveChangesAsync) ---
await _db.Productos.AddAsync(nuevo);
await _db.SaveChangesAsync();   // recién aquí se ejecuta el INSERT

_db.Productos.Update(productoModificado);
await _db.SaveChangesAsync();   // UPDATE

_db.Productos.Remove(productoAEliminar);
await _db.SaveChangesAsync();   // DELETE

// Patrón común: buscar y actualizar campos puntuales
var prodMod = await _db.Productos.FindAsync(id);
prodMod.Precio = nuevoPrecio;
await _db.SaveChangesAsync();   // EF detecta los cambios y genera el UPDATE
```

07 · Validación, errores, middleware y CORS

Data Annotations y validaciones

Son **atributos (decoradores)** que se colocan sobre las propiedades de una clase para declarar reglas de validación y configuración de base de datos. ASP.NET Core las verifica automáticamente en cada request: si algo falla, retorna un `400 Bad Request` con los errores detallados.

Clase con Data Annotations

```
public class Producto {
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio")]
    [MaxLength(100, ErrorMessage = "Máximo 100 caracteres")]
    public string Nombre { get; set; }

    [Range(0.01, 999999, ErrorMessage = "El precio debe ser positivo")]
    public decimal Precio { get; set; }

    [EmailAddress(ErrorMessage = "Email inválido")]
    public string ContactoEmail { get; set; }

    [Url(ErrorMessage = "Debe ser una URL válida")]
    public string ImagenUrl { get; set; }
}

// En el controlador: ModelState.IsValid las verifica todas
[HttpPost]
public async Task<IActionResult> Crear([FromBody] Producto nuevo) {
    if (!ModelState.IsValid)
        return BadRequest(ModelState); // 400 con lista de errores
    // ...
}
```

Annotation	Qué valida
[Required]	El campo no puede ser nulo ni vacío.
[MaxLength(n)]	La cadena no puede tener más de n caracteres.
[MinLength(n)]	La cadena debe tener al menos n caracteres.
[Range(min, max)]	El número debe estar entre min y max.
[EmailAddress]	Debe tener formato de email válido.
[Url]	Debe ser una URL válida (http/https).
[RegularExpression]	Debe coincidir con una expresión regular.

Annotation	Qué valida
[Compare("Campo")]	Debe tener el mismo valor que otro campo (ej: contraseñas).

Manejo de errores

En una API, los errores no manejados devuelven un `500` con el stack trace al cliente (un problema de seguridad y de experiencia). El objetivo es capturar los errores, loguearlos internamente y devolver respuestas claras y seguras al cliente.

En lugar de poner `try/catch` en cada controlador, lo más limpio es registrar un **middleware global** que capture todas las excepciones no manejadas:

ExceptionMiddleware.cs — middleware global de excepciones

```
public class ExceptionMiddleware {
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(RequestDelegate next,
        ILogger<ExceptionMiddleware> logger) {
        _next = next; _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context) {
        try { await _next(context); }
        catch (KeyNotFoundException ex) {
            context.Response.StatusCode = 404;
            await context.Response.WriteAsJsonAsync(new { error = ex.Message });
        }
        catch (ArgumentException ex) {
            context.Response.StatusCode = 400;
            await context.Response.WriteAsJsonAsync(new { error = ex.Message });
        }
        catch (Exception ex) {
            _logger.LogError(ex, "Error no manejado");
            context.Response.StatusCode = 500;
            await context.Response.WriteAsJsonAsync(new { error = "Error interno" });
        }
    }
}

// Program.cs: registrar antes de los demás middleware
app.UseMiddleware<ExceptionMiddleware>();
```

Middleware

Un middleware es un componente que se ubica en el **pipeline** de ASP.NET Core y procesa cada request HTTP antes de que llegue al controlador, y cada response antes de que salga al cliente. Cada middleware puede decidir: procesar el request, pasarlo al siguiente, o cortarlo y retornar una respuesta directamente. **El orden importa.**

Pipeline de middleware — el orden importa

```
var app = builder.Build();

app.UseMiddleware<ExceptionMiddleware>(); // 1ro: captura errores
app.UseHttpsRedirection();                // 2do: fuerza HTTPS
app.UseAuthentication();                  // 3ro: valida quién sos (JWT)
app.UseAuthorization();                   // 4to: valida qué podés hacer
app.MapControllers();                     // 5to: enruta al controlador

// La response viaja de vuelta en orden INVERSO
```

Estructura interna de un middleware

```
public class MiMiddleware {
    private readonly RequestDelegate _next;
    public MiMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context) {
        // Código ANTES de pasar al siguiente middleware
        Console.WriteLine($"Request: {context.Request.Path}");
        await _next(context); // pasar al siguiente
        // Código DESPUÉS (cuando la response ya fue generada)
        Console.WriteLine($"Response: {context.Response.StatusCode}");
    }
}
```

Middleware	Controlador
Se ejecuta en TODOS los requests	Se ejecuta solo cuando la ruta coincide
Ideal para: logging, auth, CORS, compresión, errores globales	Ideal para: lógica de negocio de un endpoint específico
No conoce los parámetros de la ruta	Tiene acceso a parámetros, body, query string
Registrado en Program.cs con `app.Use...()`	Registrado con atributos `[HttpGet]`, `[HttpPost]`...

CORS — Cross-Origin Resource Sharing

CORS es un mecanismo de seguridad del **navegador** que bloquea requests HTTP hechos desde un dominio distinto al de la API. Por ejemplo, si tu frontend corre en `localhost:3000` y tu API en `localhost:5000`, el navegador bloquea el request a menos que la API declare explícitamente que ese origen está permitido.

> **Ojo:** El error CORS **no lo genera tu API** — lo genera el navegador como medida de seguridad. Por eso aparece en el browser pero no en Postman (que no es un navegador y no aplica la política).

Antes de un POST, PUT o DELETE, el navegador envía automáticamente un request previo con el verbo `OPTIONS` (el **preflight**) para preguntarle al servidor si el origen tiene permiso. Si el servidor responde con

los headers correctos, recién entonces envía el request real.

Configuración de CORS en Program.cs

```
// Política permisiva para desarrollo
builder.Services.AddCors(options => {
    options.AddPolicy("DesarrolloPolicy", policy => {
        policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
    });
});

// Política restrictiva para producción (recomendada)
builder.Services.AddCors(options => {
    options.AddPolicy("ProduccionPolicy", policy => {
        policy.WithOrigins("https://mifrontend.com")   // solo estos orígenes
              .WithMethods("GET", "POST", "PUT", "DELETE")
              .WithHeaders("Content-Type", "Authorization")
              .AllowCredentials();                     // permite cookies/auth
    });
});

var app = builder.Build();
// IMPORTANTE: UseCors debe ir ANTES de UseAuthentication y MapControllers
app.UseCors("DesarrolloPolicy");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Error / Situación	Causa	Solución
El error aparece en el browser pero no en Postman	Postman no es un browser y no aplica la política CORS	Configurar CORS en la API correctamente
`AllowAnyOrigin()` con `AllowCredentials()`	No se pueden combinar — restricción de seguridad del estándar	Usar `WithOrigins()` explícito
`UseCors()` después de `MapControllers()`	El middleware de CORS no llega a ejecutarse a tiempo	Mover `app.UseCors()` antes de `app.MapControllers()`
El frontend usa http pero la API es https	Origen diferente por protocolo — http y https son distintos orígenes	Agregar ambas variantes en `WithOrigins()`

08 · Programación asíncrona y el CLR

async / await

La programación asíncrona permite que un hilo **no quede bloqueado** esperando una operación lenta (consulta a BD, llamada HTTP, lectura de archivo). En lugar de esperar parado, el hilo queda libre para atender otros requests. Cuando la operación termina, el resultado se retoma. En .NET esto se logra con `async`, `await` y `Task`.

> **¿Qué es un hilo (thread)?** Es una unidad de ejecución del procesador. Un servidor web tiene un *pool* de hilos limitado. Si cada request bloquea un hilo esperando una consulta SQL, con pocos usuarios simultáneos el servidor se queda sin hilos y rechaza requests. La programación asíncrona evita este problema.

MAL — síncrono: el hilo queda bloqueado

```
public Producto GetById(int id) {
    var prod = _db.Productos.Find(id);  // hilo bloqueado aquí
    return prod;
}
```

BIEN — asincrónico: el hilo queda libre

```
public async Task<Producto> GetByIdAsync(int id) {
    var prod = await _db.Productos.FindAsync(id); // hilo libre aquí
    return prod;
}
```

`Task` representa una operación asíncrona sin valor de retorno (como `void`). `Task<T>` representa una operación que retorna un valor de tipo T. Son el tipo de retorno obligatorio de cualquier método `async`.

> **Ojo: Nunca** uses `.Result` ni `.Wait()` — causan deadlocks en ASP.NET Core. Y **nunca** uses `async void` en APIs — los errores no se propagan. Usá siempre `await` y `async Task`.

CLR — Common Language Runtime

El CLR es la **máquina virtual** de .NET: el motor que ejecuta tu código C#. Se encarga de compilar, gestionar la memoria, manejar errores y garantizar la seguridad en tiempo de ejecución. Gracias al CLR, el mismo código compilado puede correr en Windows, Linux y macOS sin cambios.

Responsabilidad del CLR	Qué significa en la práctica
Compilación JIT	Tu C# se compila a IL (Intermediate Language) y el CLR lo convierte a código nativo de la CPU en tiempo de ejecución. Por eso .NET corre en cualquier plataforma.

Responsabilidad del CLR	Qué significa en la práctica
Garbage Collector (GC)	Administra la memoria automáticamente. Cuando un objeto no tiene referencias, el GC lo detecta y libera la memoria. No necesitás hacer `free()` como en C++.
Manejo de excepciones	Provee el mecanismo de `try/catch/finally` de forma uniforme en todo el runtime.
Seguridad de tipos	Garantiza que no podés hacer operaciones inválidas entre tipos. Un `string` no puede tratarse como `int` sin conversión explícita.
Gestión de hilos	Administra el Thread Pool que usa ASP.NET Core para procesar requests simultáneos con async/await.

El viaje de tu código C# hasta la CPU

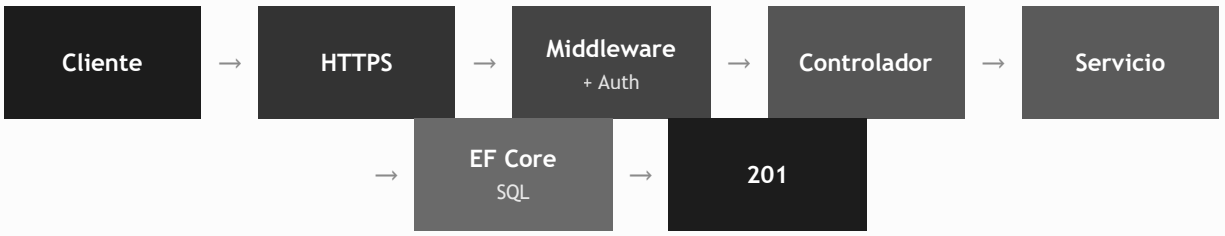
```
// 1. Escribís código C#
public int Sumar(int a, int b) => a + b;

// 2. El compilador Roslyn lo convierte a IL (independiente de plataforma)
// 3. Al ejecutar, el CLR compila el IL a código nativo (JIT)
//    Primera ejecución: se compila. Siguientes: usa la versión en caché.
// 4. El GC monitorea la memoria durante toda la ejecución
var lista = new List<Producto>(); // CLR asigna memoria en el heap
lista = null;                     // sin referencias -> el GC la libera
// 5. En ASP.NET Core, el CLR gestiona el Thread Pool
```

09 · Flujo completo de un request

Juntemos todo. Este es el recorrido de un `POST /api/productos` con JSON y JWT, desde que el cliente lo envía hasta que recibe la respuesta, pasando por todas las capas de una API .NET con EF Core.

Paso	Capa	Responsabilidad
1	Cliente	Envía POST /api/productos con JSON y JWT en el header
2	HTTPS / TLS	El mensaje viaja cifrado por TLS
3	CLR / Thread Pool	Asigna un hilo del pool para el request
4	ExceptionMiddleware	Envuelve todo — captura cualquier error no manejado
5	UseAuthentication	Valida el JWT — si falla: 401
6	UseAuthorization	Chequea permisos — si falla: 403
7	Controlador	Valida Data Annotations (ModelState) — si falla: 400. Decide qué IActionResult retornar
8	Servicio	async/await: hilo libre mientras aplica la lógica de negocio
9	Repositorio	Llama al DbContext de EF Core
10	EF Core / LINQ	Traduce las lambdas a SQL
11	SaveChangesAsync()	Recién aquí EF ejecuta el SQL real en la BD
12	CLR / GC	Libera objetos temporales, devuelve el hilo al pool
13	Cliente	Recibe JSON — status 201 Created



El request atraviesa cada capa y la respuesta vuelve en orden inverso.

10 · Ejercicios prácticos

Los ejercicios están organizados en tres niveles: **Comprensión** (preguntas conceptuales), **Código** (escribir o corregir código) y **Desafío** (integrar varios conceptos). Se recomienda resolverlos sin mirar el material antes de consultar.

Nivel 1 — Comprensión

- **1.** ¿Cuál es la diferencia entre HTTP y HTTPS? ¿Por qué HTTPS es obligatorio en producción?
- **2.** Explicá con tus palabras qué diferencia hay entre PUT y PATCH. ¿Cuándo usarías cada uno?
- **3.** ¿Por qué el controlador devuelve `IActionResult` en lugar de devolver el objeto directamente?
- **4.** Un cliente recibe un error 401. ¿Qué significa? ¿Es lo mismo que un 403? Explicá la diferencia.
- **5.** ¿Qué hace `SaveChangesAsync()`? ¿Qué pasaría si llamás `AddAsync()` pero olvidás llamar a `SaveChangesAsync()`?
- **6.** ¿Qué es el Garbage Collector del CLR? ¿Por qué en C# no necesitás liberar memoria manualmente?
- **7.** Explicá qué hace `await` en un método `async`. ¿Qué le pasa al hilo mientras espera?
- **8.** ¿Cuál es la diferencia entre `AddScoped`, `AddSingleton` y `AddTransient`? Dá un ejemplo de cuándo usar cada uno.
- **9.** ¿Qué es REST? ¿En qué se diferencia de SOAP? ¿Cuándo elegirías uno sobre el otro?
- **10.** ¿Para qué sirve una migración en EF Core? ¿Qué comando se usa para generarla?

Nivel 2 — Código

11. El siguiente código tiene un error grave. Identificalo y corregilo:

Ejercicio 11 — encontrar el error

```
public async Task<List<Producto>> ObtenerCarosAsync() {
    var todos = _db.Productos.ToList();
    return todos.Where(p => p.Precio > 1000).ToList();
}
```

12. Completá el método para que retorne 404 si el producto no existe, y 200 con el producto si existe:

Ejercicio 12 — completar

```
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id) {
    var prod = await _service.ObtenerPorIdAsync(id);
    // tu código aquí
}
```

13. Escribí la clase `Categoria` con las propiedades `Id` (int), `Nombre` (string, obligatorio, máximo 50 caracteres) y `Descripcion` (string, máximo 200 caracteres). Agregá los Data Annotations correspondientes.

14. El siguiente código causa un deadlock en ASP.NET Core. ¿Por qué? Corregilo:

Ejercicio 14 — deadlock

```
public IActionResult GetTodos() {
    var productos = _service.ObtenerTodosAsync().Result;
    return Ok(productos);
}
```

15. Escribí una consulta LINQ con EF Core que obtenga los 5 productos más caros de la categoría 'Electrónica', ordenados de mayor a menor precio, incluyendo la navegación a Categoría.

16. ¿Qué diferencia hay entre estas dos consultas? ¿Cuál es más eficiente y por qué?

Ejercicio 16 — comparar consultas

```
// Opción A
var todos = await _db.Productos.ToListAsync();
var caros = todos.Where(p => p.Precio > 1000).ToList();

// Opción B
var caros = await _db.Productos
    .Where(p => p.Precio > 1000)
    .ToListAsync();
```

Nivel 3 — Desafío

- **17.** Diseñá los endpoints REST para una API de una librería con entidades `Libro` y `Autor`. Describí la URL, el verbo HTTP, qué recibe en el body (si aplica) y qué status codes puede retornar para cada operación CRUD de ambas entidades.
- **18.** Creá un servicio completo para la entidad `Libro` con los métodos: ObtenerTodosAsync, ObtenerPorIdAsync, CrearAsync (validar que el ISBN no se repita), ActualizarAsync y EliminarAsync. Incluí el manejo de errores apropiado.
- **19.** Explicá paso a paso qué ocurre desde que un cliente envía `POST /api/libros` con un JSON y un JWT hasta que recibe la respuesta. Mencioná todas las capas: middleware, autenticación, controlador, Data Annotations, servicio, repositorio, EF Core, CLR.
- **20.** Escribí un middleware de logging que registre en consola el método HTTP, la URL, el status code de la respuesta y el tiempo que tardó en procesarse cada request. Registralo en Program.cs en el lugar correcto del pipeline.

11 · Glosario completo

Concepto	En una línea
HTTP	Protocolo que define cómo se comunican cliente y servidor.
HTTPS	HTTP + cifrado TLS. Obligatorio en producción.
REST	Estilo de arquitectura: recursos, verbos HTTP y sin estado.
SOAP	Protocolo formal con XML. Usado en sistemas empresariales.
Verbo HTTP	GET/POST/PUT/PATCH/DELETE: indica la intención del request.
Endpoint	Combinación de verbo + URL que acepta una petición.
JSON	Formato de datos liviano y legible usado en APIs REST.
Header	Metadato del request/response (auth, content-type...).
JWT	Token firmado para autenticar usuarios en cada request.
Lambda (x=>x)	Función anónima inline. Muy usada en LINQ.
Variable	Espacio en memoria con nombre y tipo.
Clase	Molde para crear objetos con propiedades y métodos.
Clase estática	No se instancia. Agrupa utilidades y constantes.
Interfaz	Contrato: define qué métodos debe implementar una clase.
Servicio	Contiene la lógica de negocio. Inyectado en el controlador.
Controlador	Punto de entrada HTTP. Retorna IActionResult.
Program.cs	Registra servicios y configura el pipeline al arrancar.
ORM	Traduce clases C# a tablas SQL automáticamente.
Entity Framework Core	El ORM oficial de .NET.
DbContext	Clase central de EF Core. Representa la sesión con la BD.
DbSet<T>	Representa una tabla de BD. Punto de entrada para LINQ.
SaveChangesAsync()	Ejecuta todos los cambios pendientes en la BD.
Migración	Archivo que describe un cambio en el esquema de la BD.
LINQ	Consultas integradas en C# que EF traduce a SQL.

Concepto	En una línea
Inyección de dependencias	Las clases reciben sus dependencias en el constructor.
Data Annotations	Atributos que declaran reglas de validación sobre propiedades.
ModelState.IsValid	Verifica que todos los Data Annotations se cumplan.
IActionResult	Interfaz que representa cualquier tipo de respuesta HTTP.
ActionResult<T>	Versión tipada de IActionResult. Recomendada en .NET moderno.
async / await	Palabras clave para programación asíncrona sin bloquear hilos.
Task / Task<T>	Tipo de retorno de métodos async. T es el valor que retorna.
Thread Pool	Conjunto de hilos reutilizables que gestiona el CLR.
CLR	Motor de ejecución de .NET: compila, gestiona memoria y hilos.
JIT	Compilación en tiempo de ejecución: IL → código nativo de la CPU.
Garbage Collector	Libera automáticamente la memoria de objetos sin referencias.
Middleware	Componente del pipeline que procesa requests y responses.
CORS	Mecanismo de seguridad del browser que controla qué orígenes pueden consumir la API.
Preflight (OPTIONS)	Request previo que el browser envía para verificar si el origen tiene permiso.
Try/Catch	Captura errores en ejecución para manejarlos correctamente.